

PB007: Software Engineering

Lecture 2: Requirements engineering

Stanislav Chren

Autumn 2026

Slido.com
#PB007



Recap

- **Software systems must be professionally developed, operated, and maintained**
- **Software engineering**
 - “**The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software**”
- **System modelling**
 - **Facilitates understanding and communication**
 - **Part of documentation**
 - **Thinking tool**

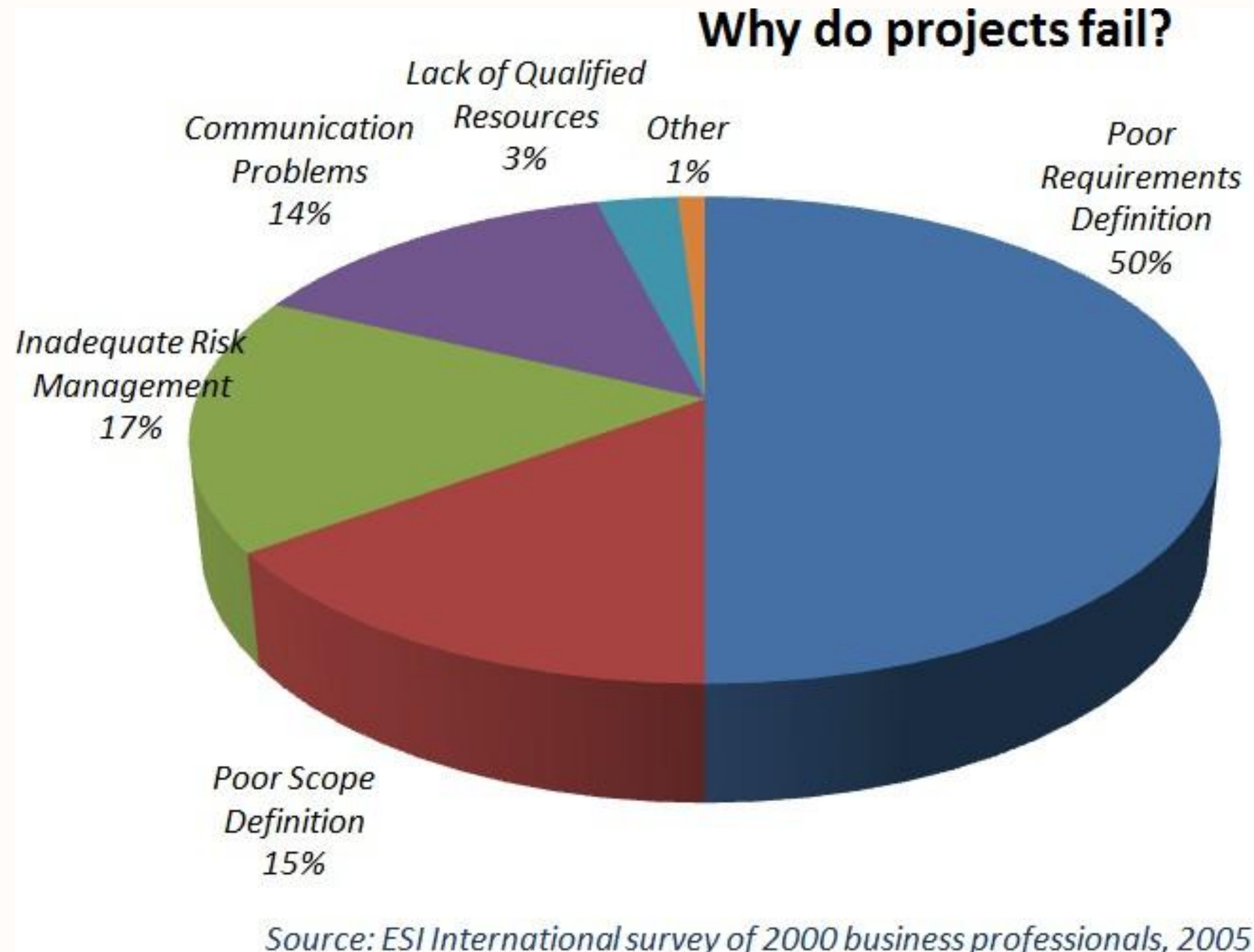
#	Knowledge Area
1	Software requirements
2	Software architecture
3	Software design
4	Software construction
5	Software testing
6	Software engineering operations
7	Software maintenance
8	Software configuration management
9	Software engineering management
10	Software engineering process
11	Software engineering models and methods
12	Software quality
13	Software security
14	Software engineering professional practice
15	Software engineering economics
16	Computing foundations
17	Mathematical foundations
18	Engineering foundations

Outline

1. Requirement types
2. Requirements engineering process
3. Non-functional requirements

1. Requirements

Motivation



Examples of Requirements Failures



- 3 → 9 years
- continuously adding new requirements
- insufficient cooperation and communication



- unification of hundreds of systems
- complicated stakeholder analysis, conflicting requirements, domain complexity, requirements prioritization,...



- 1.7 B CZK
- too general vision and procurement
- lacking technical requirements

Requirements

- ✧ **Requirements are descriptions of system services and constraints under which the system operates and is developed.**
 - **It may range from a high-level abstract statement of a service to a detailed mathematical functional specification.**
- ✧ **Requirements engineering is the process of establishing requirements.**

Functional and Non-functional Requirements

✧ Functional requirements

- **Statements of services the system provides, how the system should react to particular inputs and how the system should behave in particular situations.**

Example

A user shall be able to search the appointments lists for all clinics.

✧ Non-functional requirements

- **Properties and constraints on the services offered by the system such as timing, reliability and security constraints, constraints on the development process, platform, standards, etc.**

Example

The system shall be available on Mon–Fri, 8 am – 5 pm, with downtime not exceeding five seconds in any one day.

User vs System Requirements

✧ User requirements

- **Statements in NL + diagrams of what services the system should provide to users and operating constraints**
- **For client managers, client engineers, contractors, end-users ...**

✧ System requirements

- **more detailed, useful for implementation**
- **often part of the contract**
- **For end-users, client engineers, system architects, software developers**

User requirements definition

- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc.) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

Requirements Quality Criteria

✧ Complete

- **Are all functions required by the customer included?**

✧ Consistent

- **Are there any requirements conflicts or contradictions?**

✧ Precise

- **Is there one and only interpretation in the system context?**

✧ Verifiable

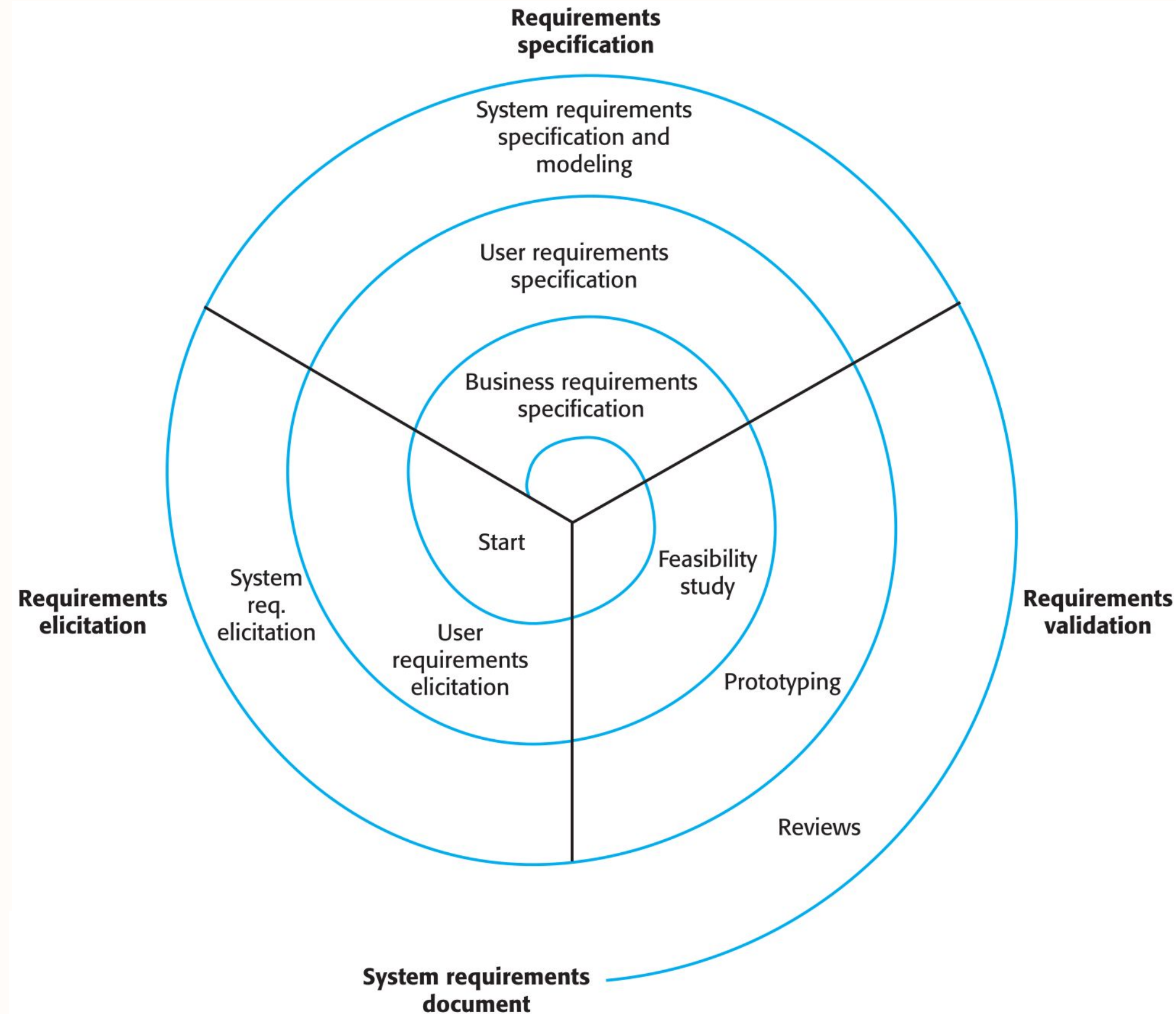
- **Can the requirements be checked?**

✧ Realistic

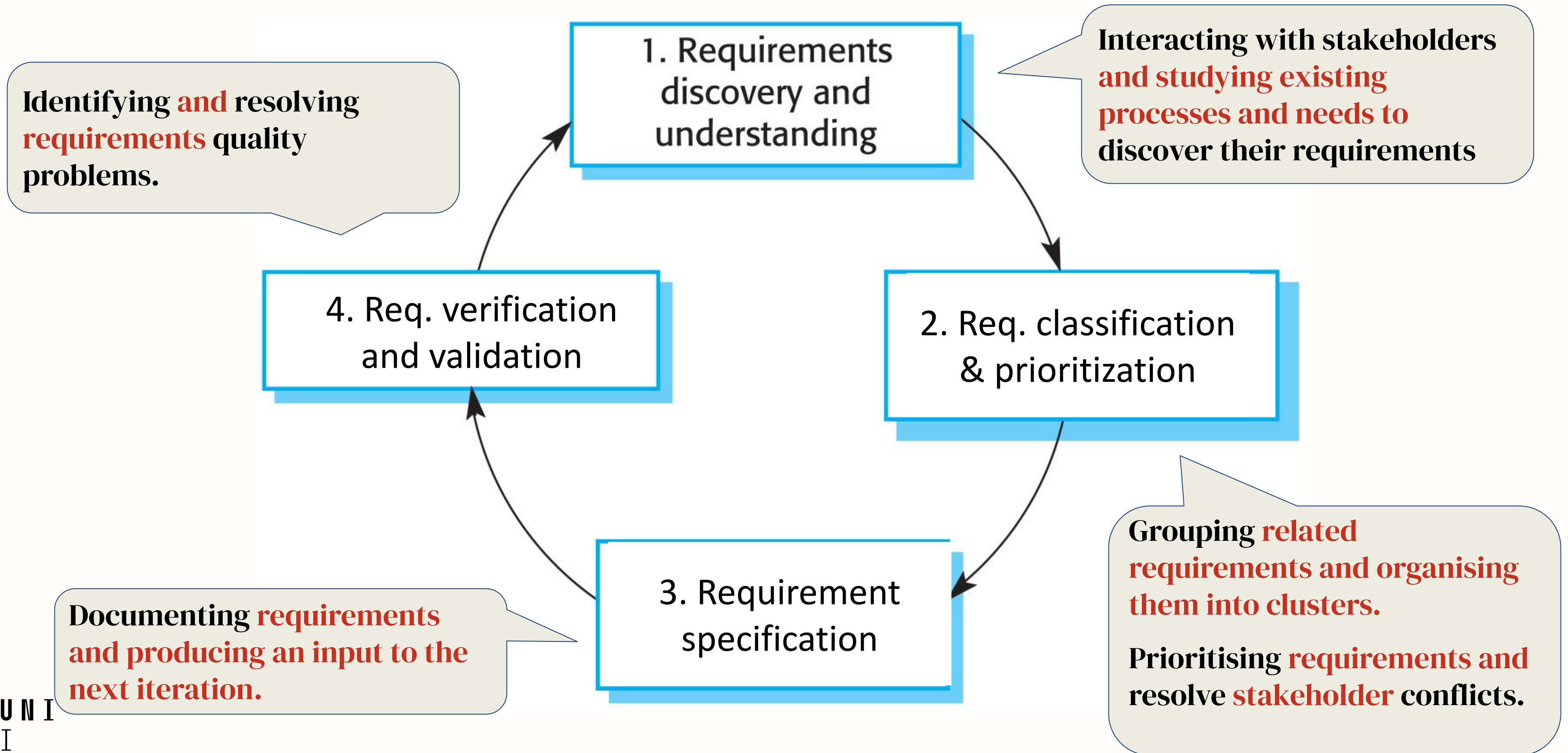
- **Can the req. be implemented with the available resources, such as budget, time and technology?**

2. Requirements engineering

The Requirements Engineering Process



The Requirements Engineering Process



Requirements discovery

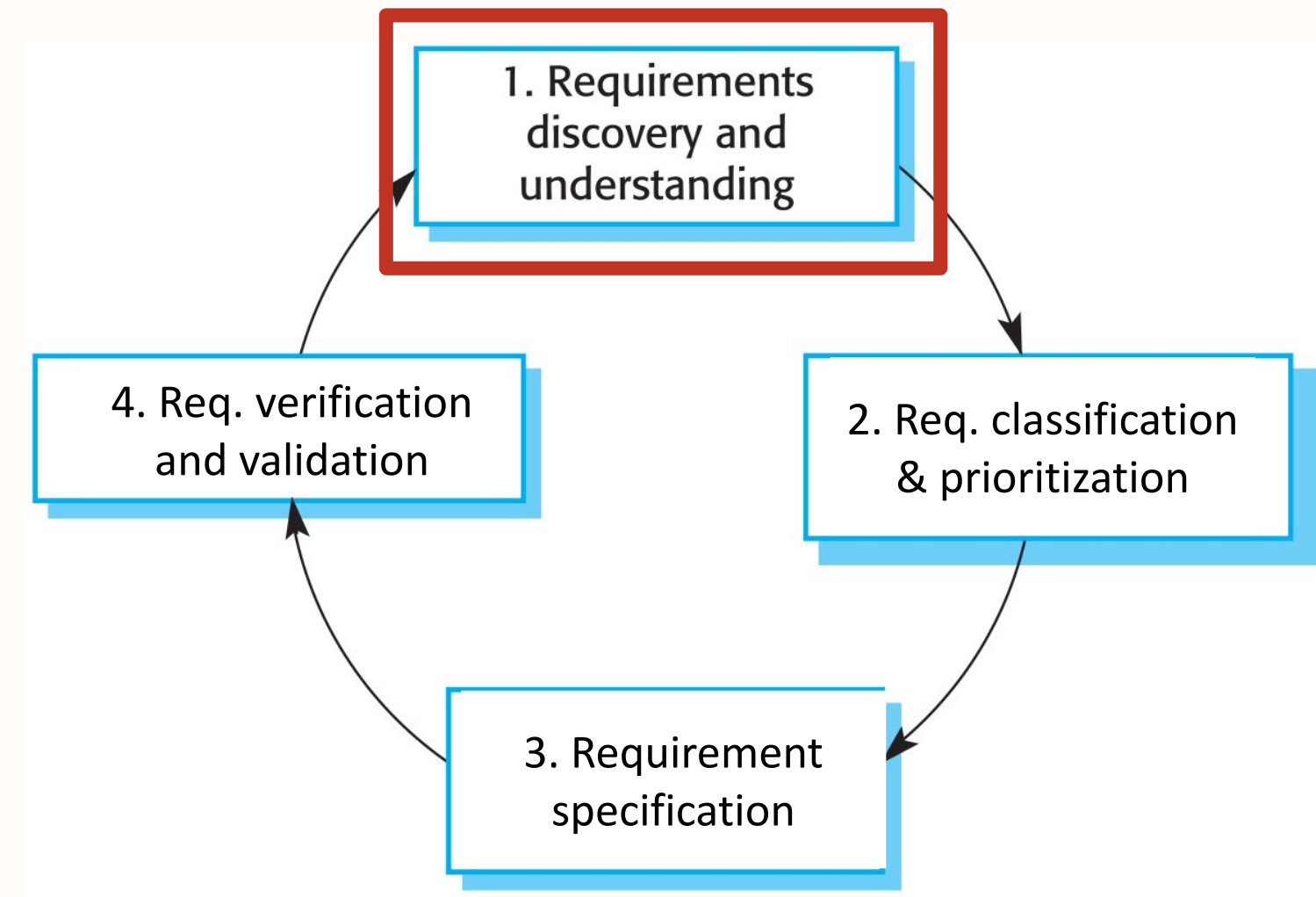
✧ Software engineers work with system stakeholders:

- end-users, managers, maintenance engineers, domain experts, trade unions, etc.

✧ To find out about:

- the application domain,
- the services to provide,
- the required system performance,
- hardware constraints,
- other systems, etc.

✧ Focus on **WHAT** the system should do **rather than** **HOW** it will do (implement) it.



Requirements discovery techniques



✧ Questionnaires

✧ Interviews

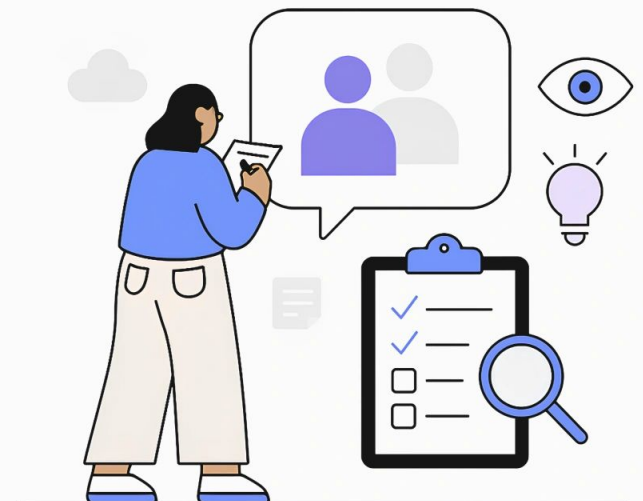
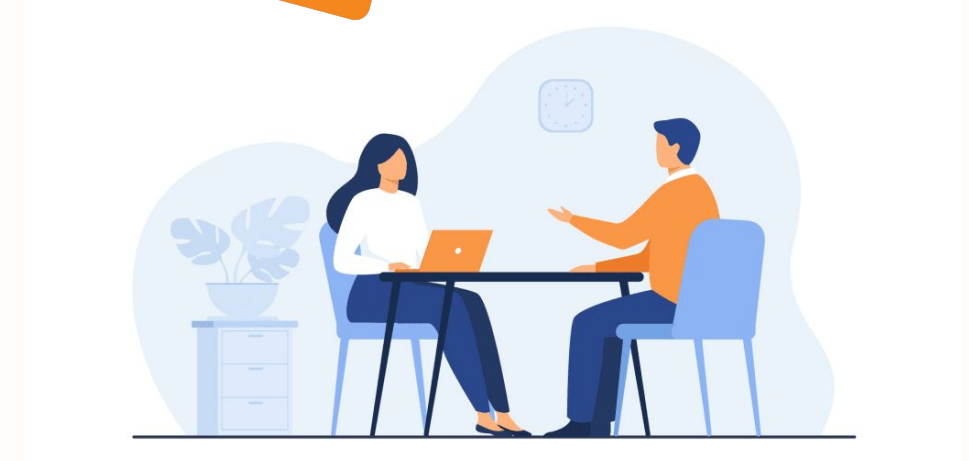
- **Small number of software engineers and stakeholders**
- **Open interviews where various issues are explored**
- **Closed interviews based on pre-determined list of questions**

✧ Workshops

- **Free brainstorming of all involved stakeholders**

✧ Ethnography

- **Observe and analyse existing processes, i.e. how people actually work, under what social and organizational factors.**



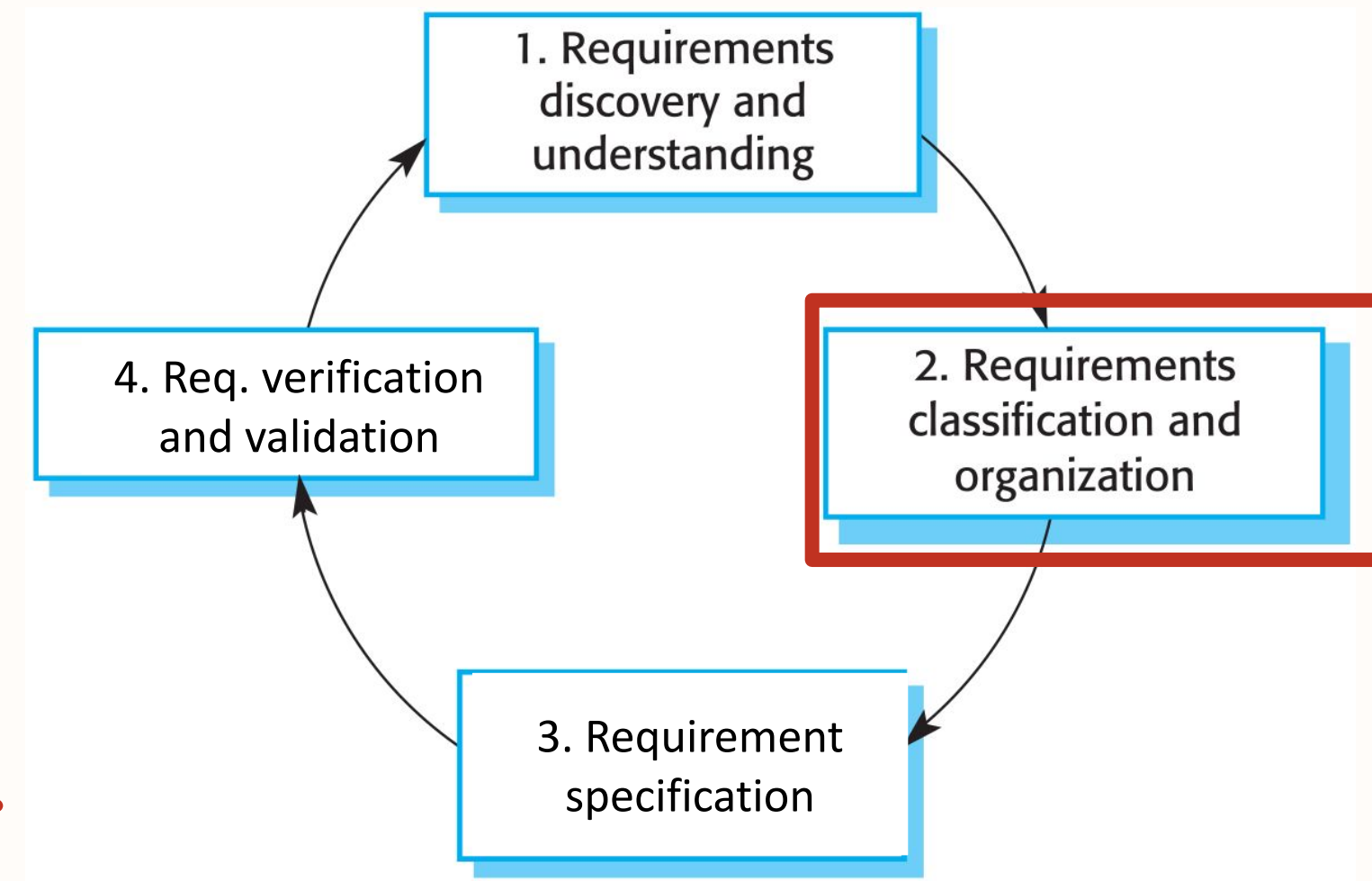
Requirements classification & prioritization

✧ MoSCoW criteria

- **Must have** – mandatory, fundamental
- **Should have** – important, may be omitted
- **Could have** – truly optional
- **Want to have** – can wait for later releases

✧ Rational Unified Process (RUP) attributes

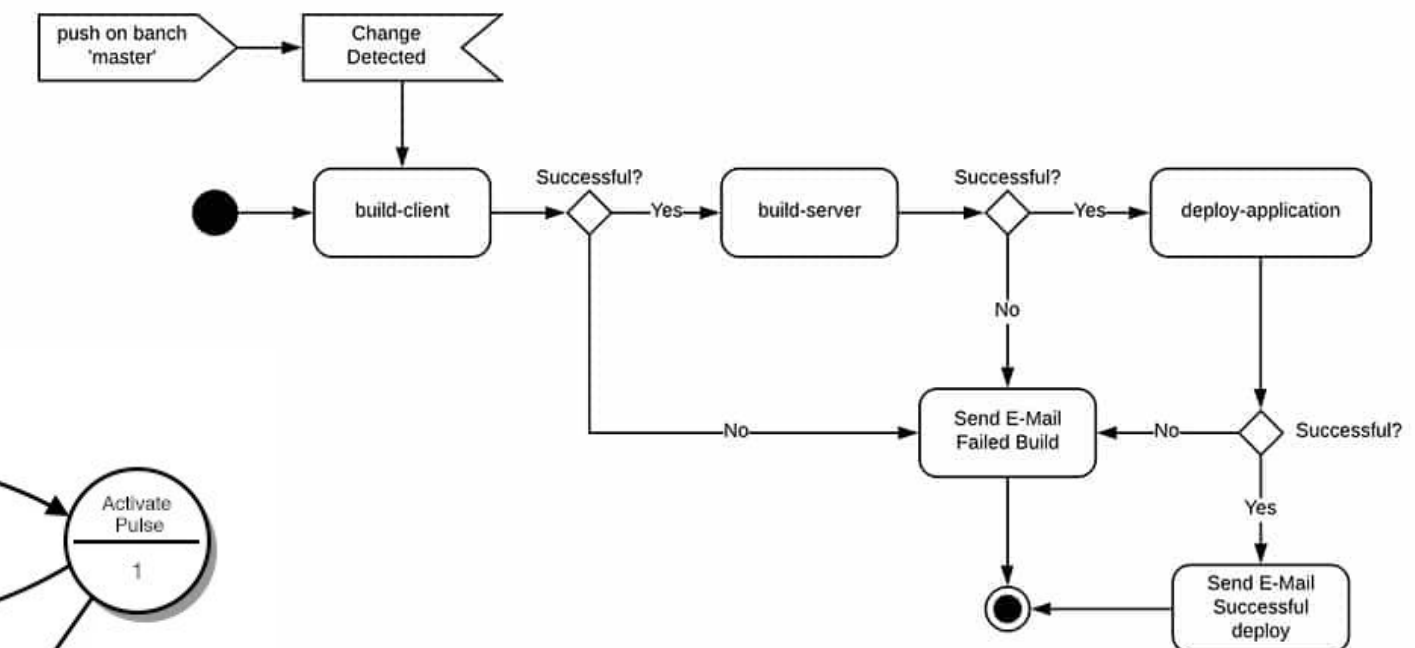
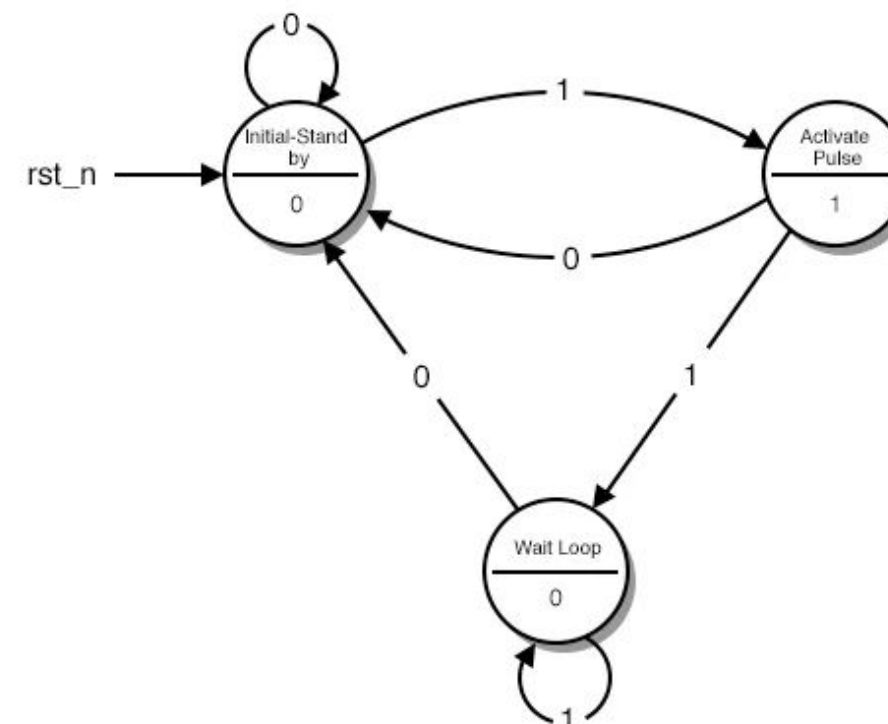
- **Status** – Proposed/Approved/Rejected/Incorporated
- **Benefit** – Critical/Important/Useful
- **Effort** – number of person days/functional points/etc.
- **Risk** – High/Medium/Low
- **Stability** – High/Medium/Low
- **Target Release** – future product version



Requirements specification

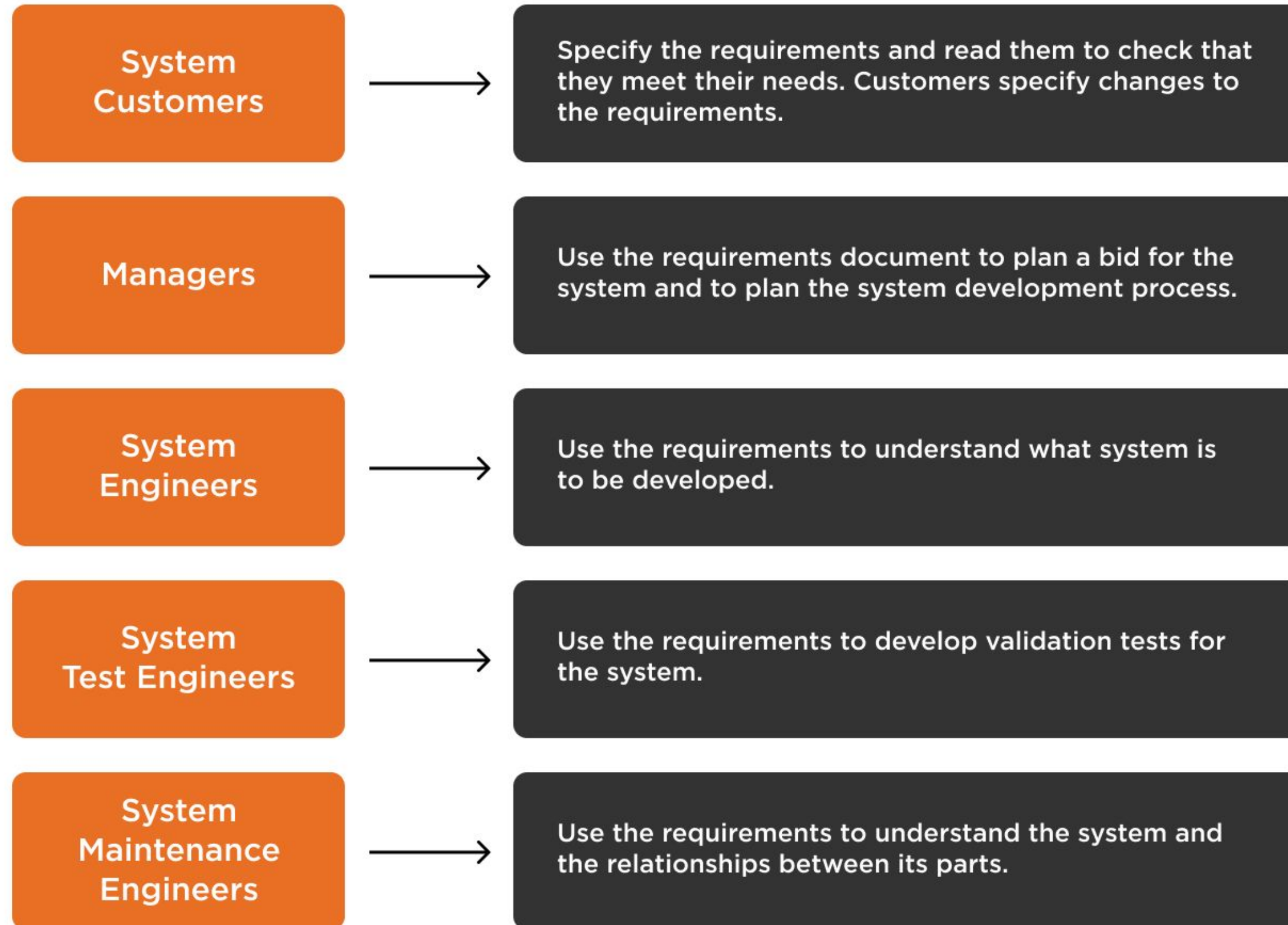
- ✧ **Natural language**
 - E.g. project assignment
- ✧ **Structured language**
 - E.g. textual spec. of UML use cases
- ✧ **Graphical notation**
 - E.g. UML Use Case or Activity diagram
- ✧ **Mathematical specification**
 - E.g. finite state machines

Use Case Name		[Name of the use case]
Actors		[An actor is a person or other entity external to the system being specified who interacts with the system and performs use cases to accomplish tasks]
Preconditions		[Activities that must take place, or any conditions that must be true, before the use case can be started]
Normal Flow	Description	[User actions and system responses that will take place during execution of the use case under normal, expected conditions.]
	Postconditions	[State of the system at the conclusion of the use case execution with a normal flow (nominal)]
Alternative flows and exceptions		[Major alternative flows or exceptions that may occur in the flow of event]
Non functional requirements		[All non-functional requirement: e.g., dependability (safety, reliability, etc.), performance, ergonomic]



Who can use requirements specification?

Who can use requirements specification?



Requirements verification & validation

✧ Requirements verification

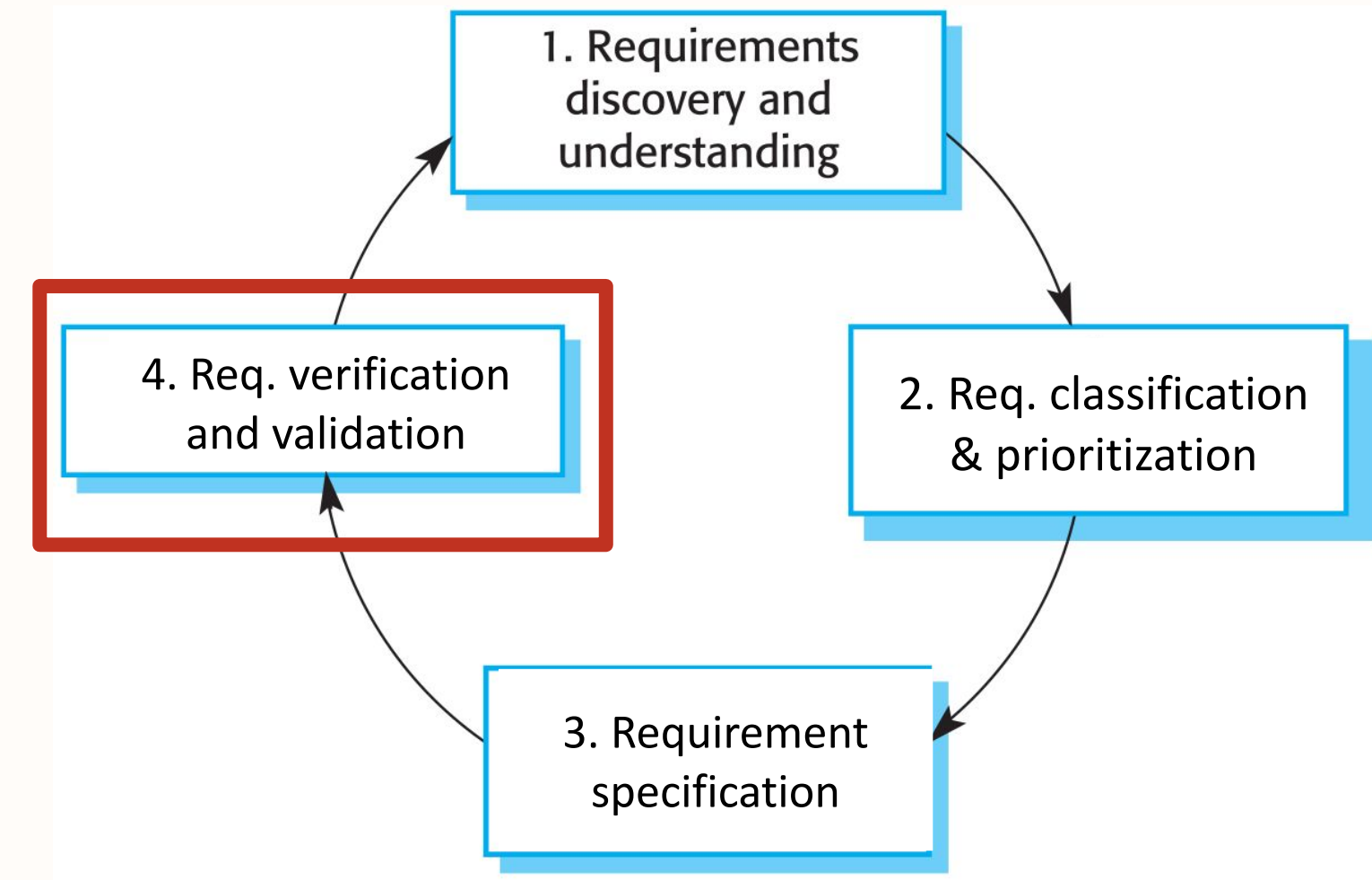
- Concerned with checking requirements completeness, consistency, preciseness, verifiability and realism.

✧ Requirements validation

- Concerned with checking that the requirements define the system that the customer really wants/needs.

✧ Techniques

- Requirements reviews
- Prototyping
- A/B testing



Requirements management and evolution

✧ Requirements management

- **Process of managing changing requirements during the requirements engineering process and system development.**
- **Each requirements change should be analysed before deciding whether to accept it.**

✧ New requirements emerge due to

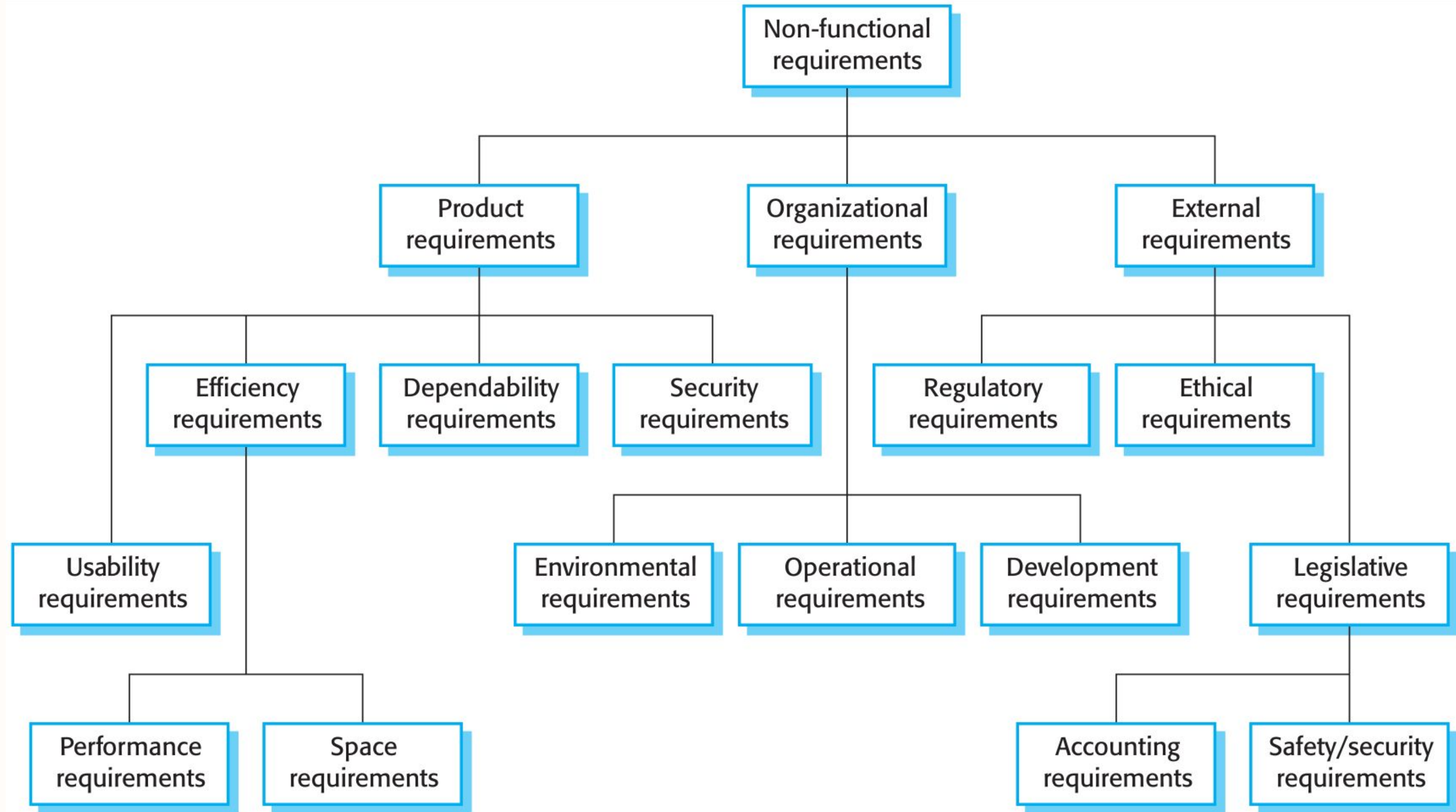
- **Business, organizational and technical changes.**

✧ Traceability

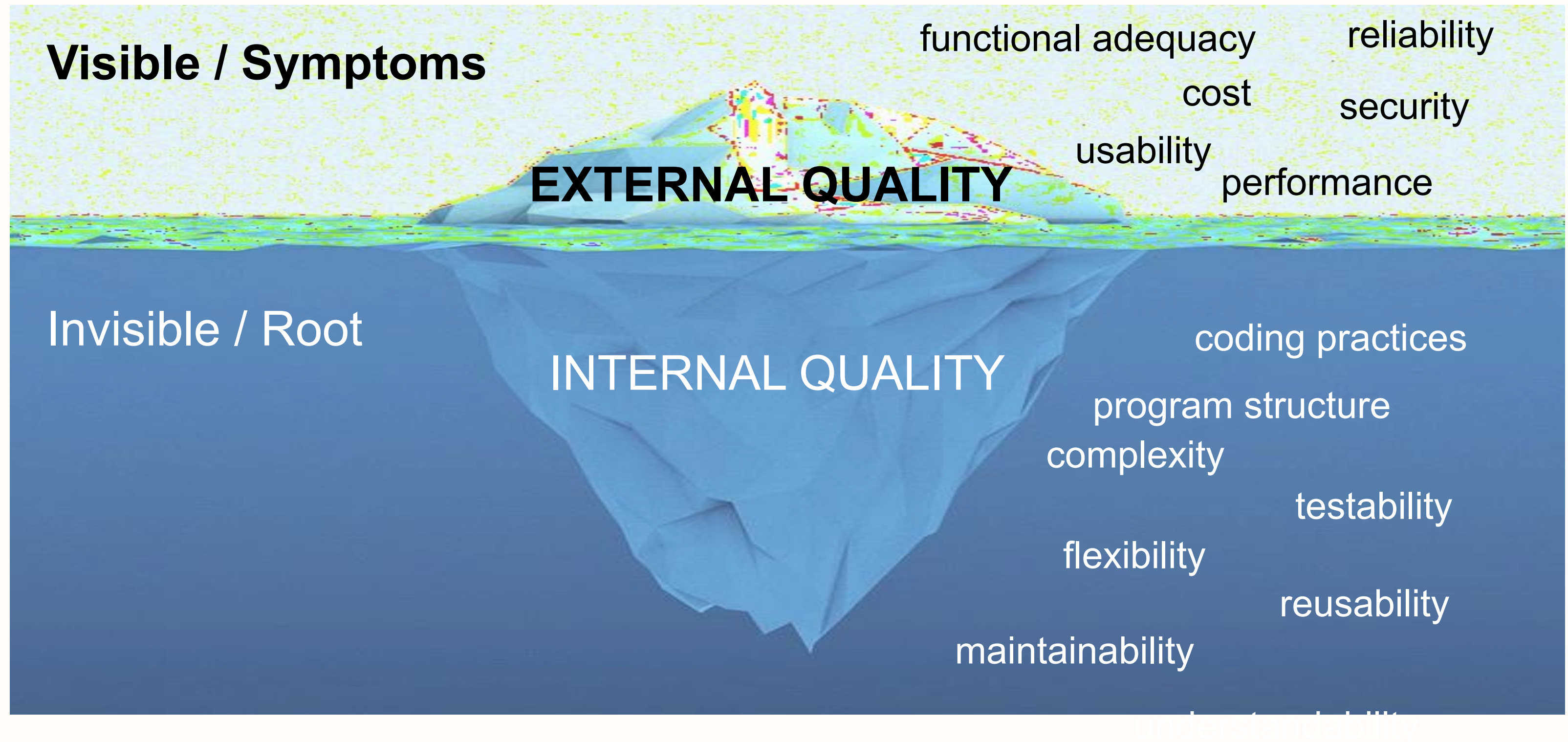
- **Links between dependent requirements**
- **Important to assess the impact of requirements changes.**

3. Non-functional Requirements

Non-functional requirements classification



Product requirements and SW Quality



Product requirements

✧ Dependability

- Availability

- Reliability

- Safety

- Security

✧ Efficiency

- Performance

- Space/resource utilization

✧ Usability

✧ Modifiability

✧ Resilience, Robustness

✧ Portability

✧ Adaptability

✧ Complexity

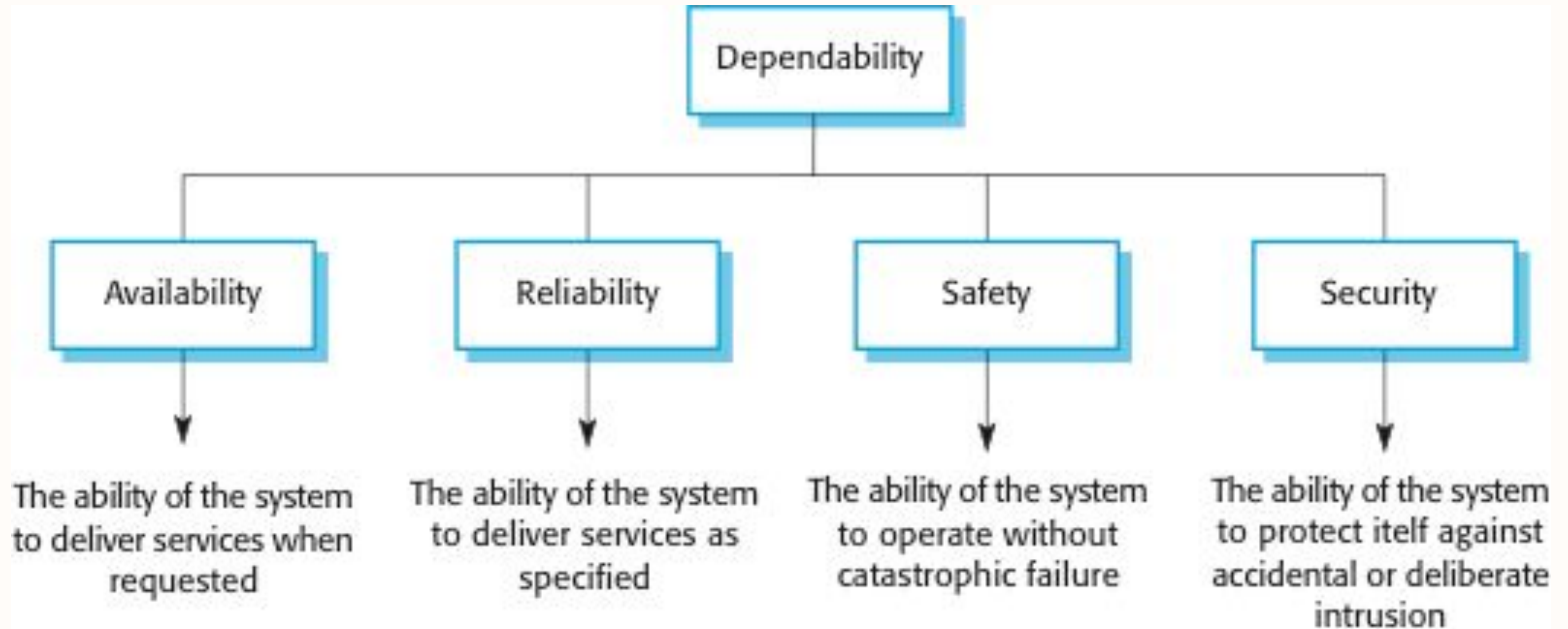
✧ Modularity, Reusability

✧ Understandability

✧ Learnability

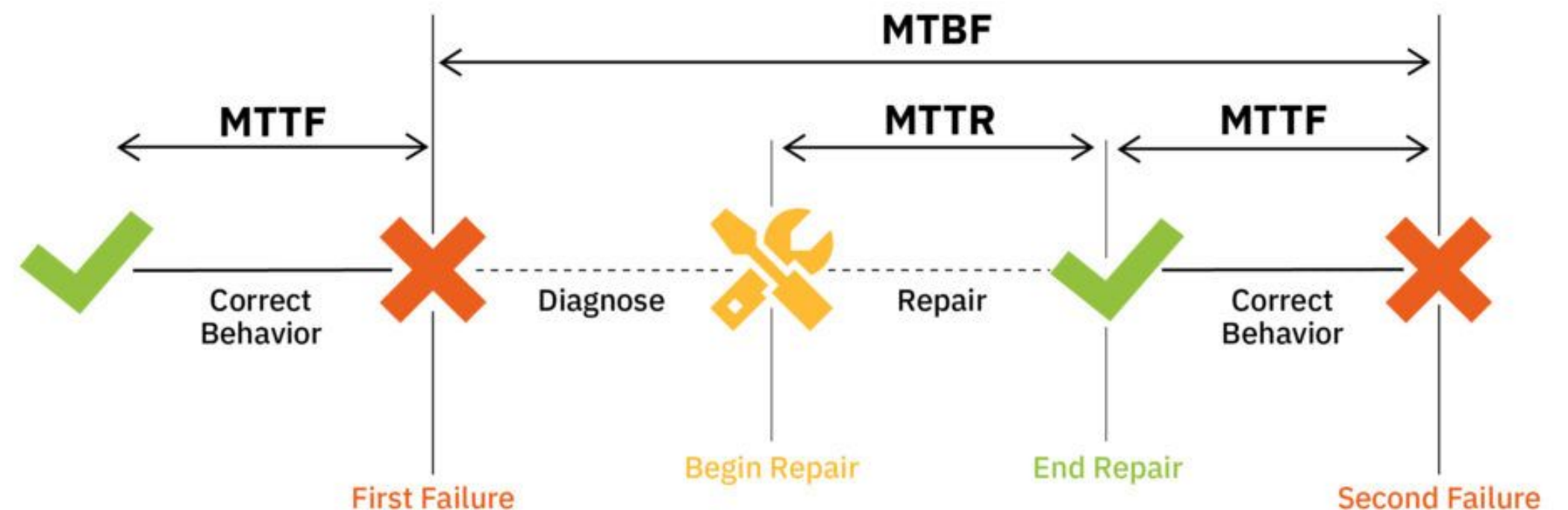
✧ Testability

Principal dependability attributes



Availability

- ✧ The probability that a system, at a point in time, will be operational and able to deliver the requested services
- ✧ Concerned with
 - How long the system should be operating without a failure.
 - How long a system is allowed to be out of operation.
- ✧ Can be expressed quantitatively
 - Using mean time to failure (MTTF) and repair (MTTR) as $\text{MTTF} / (\text{MTTF} + \text{MTTR})$.
 - I.e. availability of 0.999 means that the system is up and running for 99.9% of the time.



Reliability

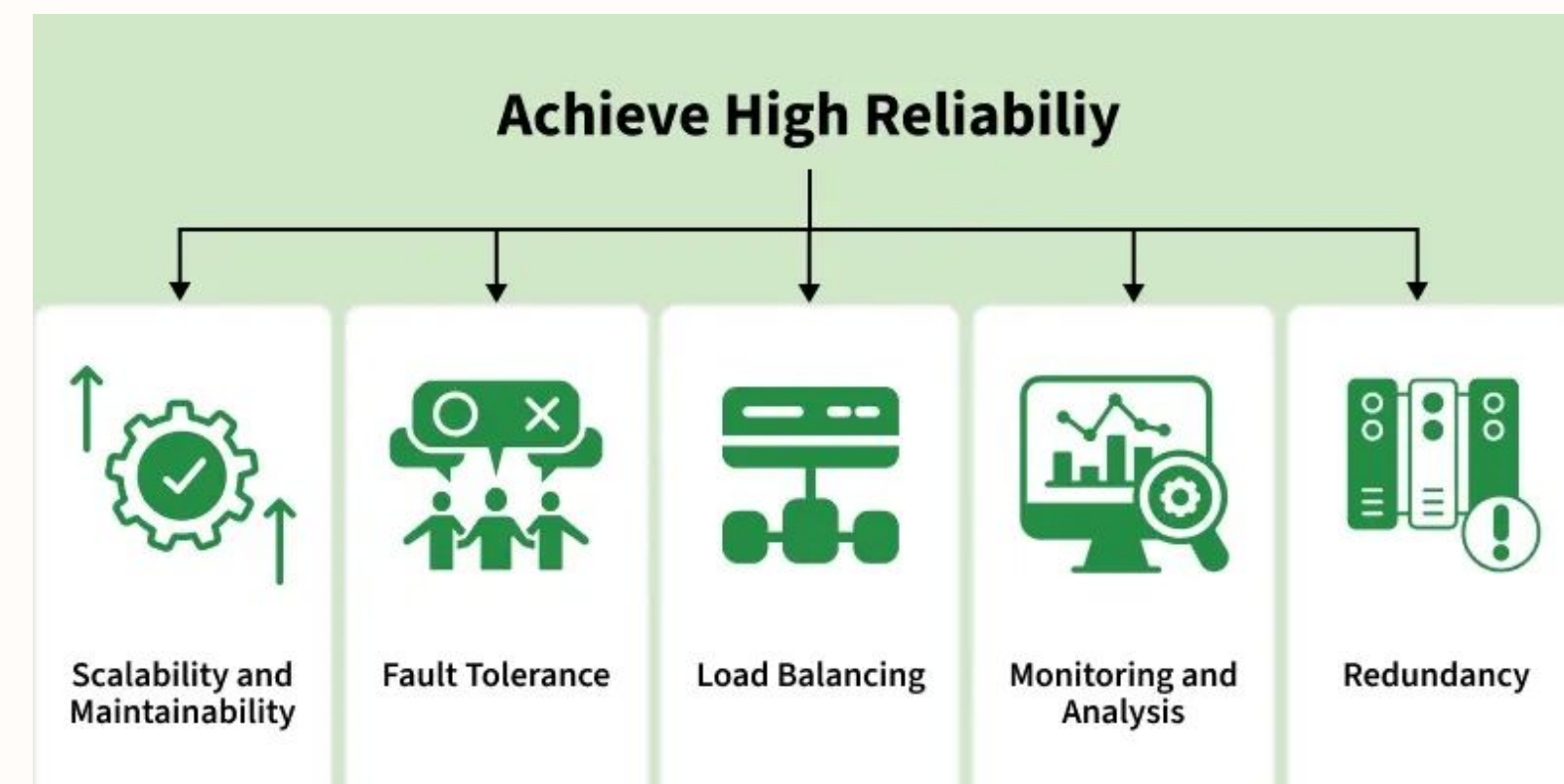
✧ The probability of failure-free system operation **over a specified time in a given environment for a given purpose**

✧ Concerned with

- How system fault/error/failure is detected.
- How frequently system fault/error/failure may occur.
- What happens when a fault/error/failure occurs.

✧ Can be expressed quantitatively

- Expressed with MTBF and MTTF in continuous operations
- Using the probability of failure on demand (PFD) within a single service or usage scenario execution.



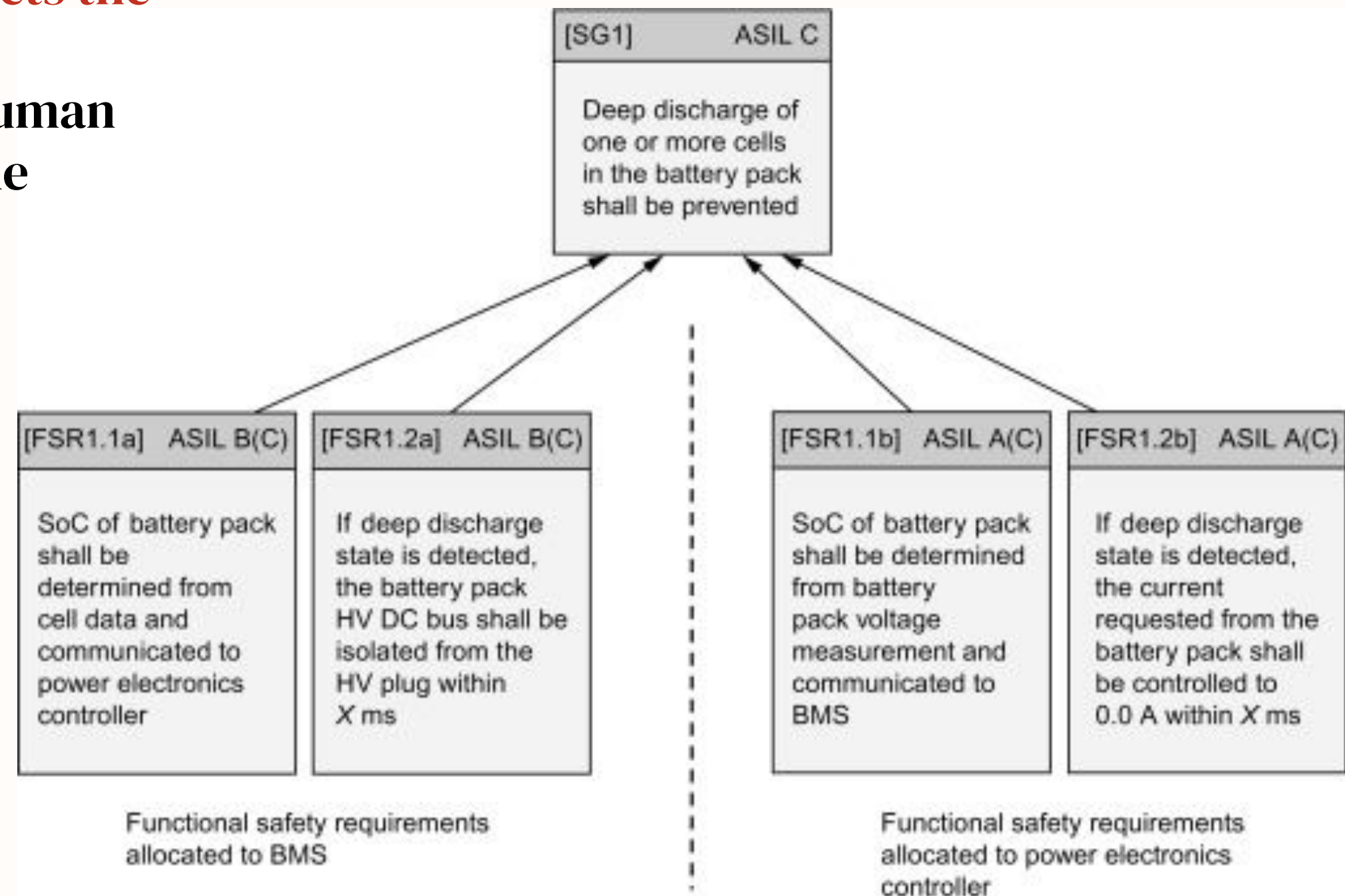
<https://www.geeksforgeeks.org/system-design/reliability-in-system-design/>

Reliability terminology

Term	Description
Human error or mistake	Human behavior that results in the introduction of faults into a system. <i>E.g., in the wilderness weather system, a programmer might decide that the way to compute the time for the next transmission is to add 1 hour to the current time. This works except when the transmission time is between 23.00 and midnight .</i>
System fault	A characteristic of a software system that can lead to a system error. <i>E.g., the inclusion of the code to add 1 hour to the time of the last transmission, without a check if the time is greater than or equal to 23.00.</i>
System error	An erroneous system state that can lead to system behavior that is unexpected by system users. <i>E.g., the value of transmission time is set incorrectly (to 24.XX rather than 00.XX) when the faulty code is executed.</i>
System failure	An event that occurs at some point in time when the system does not deliver a service as expected by its users. <i>E.g., no weather data is transmitted because the time is invalid.</i>

Safety

- ✧ Safety is a property of a system that reflects the system's ability to operate, normally or abnormally, without danger of causing human injury or death and without damage to the system's environment.
- ✧ Most devices whose failure is critical now incorporate software-based control systems.
- ✧ Safety requirements are often exclusive requirements i.e. they exclude undesirable situations rather than specify required system services.



Safety Terminology

Term	Definition
Accident (or mishap)	An unplanned event or sequence of events which results in human death or injury, damage to property, or to the environment. <i>E.g. an overdose of insulin.</i>
Hazard	A condition with the potential for causing or contributing to an accident. <i>E.g. a failure of the sensor that measures blood glucose.</i>
Damage	A measure of the loss resulting from a mishap. Damage can range from many people being killed as a result of an accident to minor injury or property damage. <i>E.g., damage resulting from an overdose of insulin could be serious injury or the death of the user of the insulin pump.</i>
Hazard severity	An assessment of the worst possible damage that could result from a particular hazard. <i>E.g. when an individual death is a possibility, a reasonable assessment of hazard severity is 'very high'.</i>
Hazard probability	The probability of the events occurring which create a hazard. Probability values tend to be arbitrary but range from 'probable' (say 1/100 chance) to 'implausible'. <i>E.g. the probability of a sensor failure in the insulin pump that results in an overdose is probably low.</i>
Risk	This is a measure of the probability that the system will cause an accident. The risk is assessed by considering the hazard probability, the hazard severity, and the probability that the hazard will lead to an accident. <i>E.g. the risk of an insulin overdose is probably medium to low.</i>

Security

- ✧ A system property that reflects the system's ability to **protect itself** from accidental or deliberate external attack.
- ✧ Defends the system against:
 - Threats to the **confidentiality** of the system and its data
 - Can disclose information to people or programs that do not have authorization to access that information.
 - Threats to the **integrity** of the system and its data
 - Can damage or corrupt the software or its data.
 - Threats to the **availability** of the system and its data
 - Can restrict access to the system and data for authorized users.

M U N I
F I Security is an essential pre-requisite for availability, reliability and safety.

Security terminology

Term	Definition
Asset	Something of value which has to be protected (<i>e.g. patients records in a health-care system</i>). The asset may be the system itself or data used by that system.
Exposure	Possible loss or harm to a computing system (<i>e.g. financial loss from patients' legal action or loss of reputation</i>). This can be loss or damage to data, or can be a loss of time and effort if recovery is necessary after a security breach.
Vulnerability	A weakness in a computer-based system that may be exploited to cause loss or harm (<i>e.g. weak password</i>).
Attack	An exploitation of a system's vulnerability. Generally, this is from outside the system and is a deliberate attempt to cause some damage.
Threats	Circumstances that have potential to cause loss or harm. You can think of these as a system vulnerability that is subjected to an attack (<i>e.g. guessing the weak password</i>).
Control	A protective measure that reduces a system's vulnerability. <i>E.g. encryption is an example of a control that reduces a vulnerability of a weak access control system, or a password checking system.</i>

Dependability Attribute Dependencies

✧ Questions

- Can a reliable system be unavailable? And vice versa?
- Can you give an example of an unreliable & safe system?
- Can you give an example of an reliable & unsafe system?

✧ Some facts about dependencies

- Safe system operation depends on the system being available and operating reliably, but not only on it.
- A system may be unreliable because its data has been corrupted by an external attack.
- Service attacks on a system are intended to make it unavailable.
- If a system is infected with a virus, you cannot be confident in its reliability or safety.

Performance

✧ Performance is about **timing** – response time to events (interrupts, messages, requests from users, or the passage of time).

- For a web-based financial system, the response might be the number of transactions that can be processed in a minute,
- or the expected duration of a single transaction (given as a random variable).

✧ Highly sensitive to concurrency effects

- number of users, shared resources,
- hardware,
- operating system implementation (e.g. scheduler strategy),...

M U N I ✧ Often accompanied by characterization of throughput and resource utilization.

F I

Requirement#	Requirement Type	Event/Use Case#
35	Performance	10
Description		
Operation Type. Any inquiry shall complete the display of its results		
Fit Criteria		
○ (Tolerable length of time) In no longer than 4 seconds ○ (Timing boundary start) From the time the user submits the request ○ (Timing boundary finish) Until the system displays the results ○ (Degree of fulfillment) For at least 99% of the inquiries submitted each hour ○ (Indicative hardware set-up) When using a 1Mbit DSL internet connection ○ (Condition of the system) While the system serves 200 simultaneous users, 5000 unique users per hour and system replication is copying the system state.		
Rationale		
○ Justification of values. This figure is based on acceptance tests of previous version of system indicating that users begin to lose patience soon after this time. ○ Exception Case (High load caveat). This requirement does not apply to inquiries across large volumes of data where arbitrary selection criteria are allowed ○ Motivation. To ensure customers do not lose patience waiting for the system's response		
Source		
Historical acceptance tests of previous version of system		
Customer Satisfaction		Customer Dissatisfaction
4		4
Dependencies		
None		
Supporting Materials		
Acceptance test results of previous version of system		
History		
This requirement was first raised by Product Manager on 12/02/2008		

Wohlrab, Rebekka et al. "Experience of pragmatically combining RE methods for performance requirements in industry." 2014 IEEE 22nd International Requirements Engineering Conference (RE) (2014): 344-353.

Usability

- ✧ **Usability is concerned with how easy it is for the user to accomplish a desired task and the kind of user support the system provides.**
- ✧ **It can be broken down into the following areas:**
 - Learning system features.
 - Using a system efficiently.
 - Minimizing the impact of errors.
 - Adapting the system to user needs.
 - Increasing confidence and satisfaction.
- ✧ **Always follow Human-Interface Guidelines (HIG) if available**
 - Windows HIG, Mac OS HIG, and others

Modifiability/Maintainability

- ✧ **Modifiability is about the cost of change.**
- ✧ **What can change (the artifact)?**
 - **functions that the system computes,**
 - **platform the system exists on (the hardware, operating system, middleware, etc.),**
 - **environment within which the system operates, etc.**
- ✧ **When is the change made and who makes it (the environment)?**
 - **During implementation (by modifying the source code), compile (using compile-time switches),**
 - **build (by choice of libraries),**
 - **configuration setup (by a range of techniques, including parameter setting),**
 - **execution (by parameter setting).**
 - **By a developer, an end user, or a system administrator.**

Organisational requirements

✧ Development requirements

- Programming language, development environment, process standards, time to market, rollout schedule, costs, etc.

✧ Operational requirements

- Execution platform and other restrictions, system usage, projected lifetime, etc.

✧ Environmental requirements

- Integration with legacy systems, targeted market, etc.

External requirements

- ✧ Regulatory requirements
- ✧ Ethic requirements
- ✧ Legislative requirements
 - Accounting legislative
 - Safety/Security legislative
 - Privacy
 - ...

Non-functional req. implementation

- ✧ **Non-functional requirements may affect the overall architecture of a system rather than the individual components.**
 - E.g., to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- ✧ **A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.**
 - It may also generate requirements that restrict existing requirements.

Key points

- ✧ **Requirements = services + constraints**
- ✧ **Requirements engineering is an iterative activity.**
 - Requirements discovery, prioritization, specification, verification & validation, management & evolution.
- ✧ **Non-functional quality attributes help us to define the quality of the software product.**
 - **Visible** – e.g. availability, reliability, safety, security, performance.
 - **Invisible** – e.g. modifiability, testability.
- ✧ **Consider non-functional requirements right from the beginning of your software project**